# Chapter 4: Neural Networks

## Classifying Structural Defects

Dr. Sam Smith

# Outline

- Introduction to neural networks
- Building blocks: neurons, activations, layers
- Classification vs. regression
- Concrete crack detection dataset
- Training strategy: mini-batch SGD, cross-entropy loss
- FNN and CNN architectures
- Evaluation metrics: accuracy, precision, recall
- Results and regularization techniques

# What Are Neural Networks?

- Computational models loosely inspired by biological neural systems.
- Organized in **layers** that transform inputs into outputs through weighted sums and nonlinear activation functions.
- During training, the model *learns* by adjusting weights to minimize a loss function.

**Brief history:**

- 1958: Perceptron (Rosenblatt) — linear decision boundaries only.
- 1980s: Backpropagation revives multi-layer networks.
- 2012+: Deep learning era — breakthroughs in image classification (AlexNet, ImageNet).

# When to Use Neural Networks

Neural networks are particularly useful when:

- The input–output relationship is **complex and nonlinear**.
- Input data is **high-dimensional** (e.g. images with thousands of pixels).
- Sufficient **labeled data** is available for training.
- An interpretable closed-form equation is **not required**.

Unlike symbolic regression (Chapter 3), neural networks are "**black boxes**" — accurate predictions, but internal mechanisms are not easily interpretable.

# The Artificial Neuron

A single neuron with $n$ inputs computes:

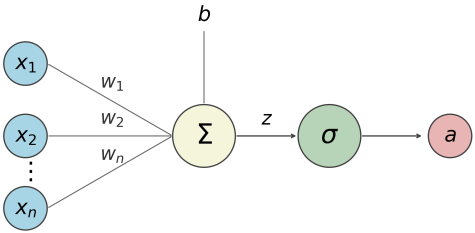
$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = \sigma(z)$$

**Weights $w_i$** Learned connection strengths.

**Bias $b$** Learned offset.

**Activation $\sigma$** Nonlinear function.




$$z = \sum_i w_i x_i + b, \quad a = \sigma(z)$$

A single artificial neuron.

# Activation Functions

**Sigmoid:**

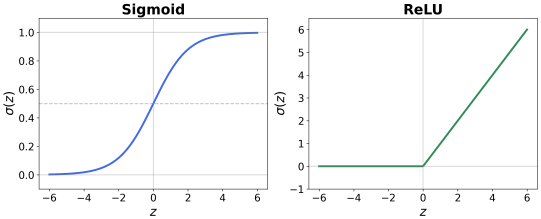
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Maps to $(0, 1)$. Historically popular but suffers from *vanishing gradients* in deep networks.

**ReLU (Rectified Linear Unit):**

$$\sigma(z) = \max(0, z)$$

Computationally efficient; default choice in modern architectures.

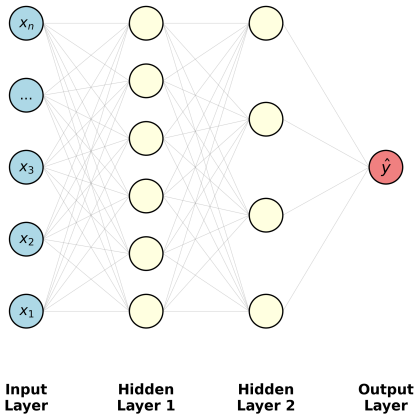



Sigmoid vs. ReLU.

# Fully Connected Feedforward Network (FNN)

Three types of layers:

- **Input layer:** raw features (flattened for images).
- **Hidden layers:** weighted sums + ReLU; every neuron connects to all neurons in the previous layer.
- **Output layer:** maps to prediction (sigmoid for binary classification).




FNN / MLP architecture.

# Classification vs. Regression

Regression (Chapters 2–3): Predict a *continuous* output (stress, deflection).

Classification (This chapter): Assign an input to one of several *discrete categories*.

**This chapter:** Binary classification — determine whether a concrete surface image contains a *crack* or *no crack*.

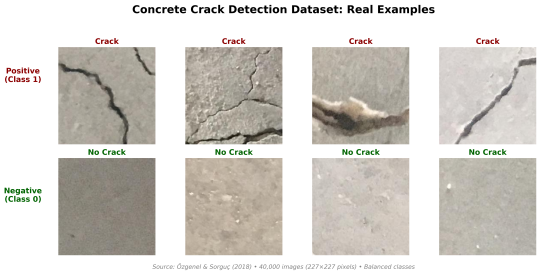
# Problem Motivation

- Cracks in concrete structures (bridges, buildings, dams) indicate degradation and potential safety hazards.
- Traditional inspection: trained personnel visually examining surfaces — time-consuming, subjective, costly.
- **Image-based ML** offers a scalable alternative for rapid screening.

# Dataset

**Concrete Crack Images for Classification** (Özgenel & Sorguç, 2019):

- 40,000 images (balanced: 20K crack, 20K no-crack).
- $227 \times 227$ pixels, RGB.
- For fast runs: 20% subset (8,000 images), 80/20 train/val split.
- Resized to $96 \times 96$ for training.

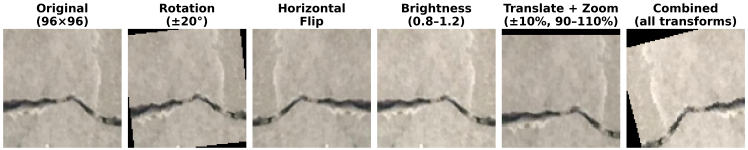



Top: cracks. Bottom: intact surfaces.

# Data Preparation

**Normalization:** $x_{\text{normalized}} = x_{\text{raw}}/255$ (maps pixels to $[0, 1]$).

**Data augmentation:** Expand the training set with random transformations that preserve the label.




Rotation, flip, brightness jitter, translation/zoom, and composite.

# Mini-Batch SGD and Adam

**Full-batch GD** (Chapter 2): use all $N$ samples per update — precise but expensive.

**Mini-batch SGD:** use a subset of $B$ samples per update: $\theta \leftarrow \theta - \eta \cdot \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta J_i(\theta)$

- Cheaper updates, stochasticity helps escape saddle points. Typical $B$: 32–256.

**Adam optimizer:** adaptive per-parameter learning rates based on gradient moments — faster and more stable convergence.

**Epochs:** one complete pass through the training dataset. Training runs for multiple epochs until convergence.

# Loss Function: Binary Cross-Entropy

For binary classification, we use the **binary cross-entropy** (log loss):

$$J = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

- $y_i \in \{0, 1\}$: true label. $\hat{y}_i \in (0, 1)$: predicted probability.
- Penalizes confident wrong predictions heavily: if $y_i = 1$ but $\hat{y}_i \approx 0$, loss $\to \infty$.
- In PyTorch: `BCEWithLogitsLoss` combines sigmoid + BCE for numerical stability.

# FNN Architecture

Each $96 \times 96 \times 3$ image is **flattened** to a vector of 27,648 values.

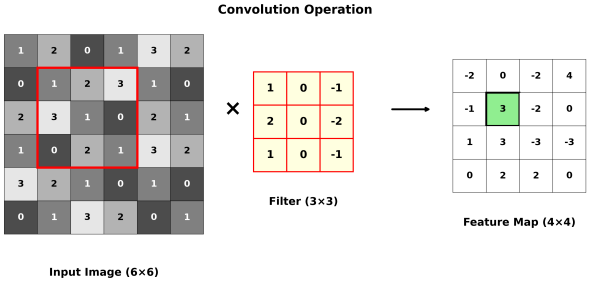
| Layer | Input dim. | Output dim. | Total params |
|---|---|---|---|
| Dense 1 (ReLU) | 27,648 | 22 | 608,278 |
| Dense 2 (ReLU) | 22 | 64 | 1,472 |
| Output (logit) | 64 | 1 | 65 |
| **Total** | | | **609,815** |

Flattening **discards spatial structure** — the network does not know which pixels are neighbors.

# Why CNNs?

- In images, meaningful patterns (edges, textures, cracks) are defined by **local spatial relationships**.
- CNNs use small learnable **filters (kernels)** that slide across the image:
  - Preserve spatial relationships.
  - Dramatically reduce parameters via sparse connectivity and weight sharing.
  - Promote translation-equivariant features.

# The Convolution Operation




For a 2D input $X$ and filter $W$ $(k \times k)$: $$Z_{i,j} = \left( \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} W_{m,n} \cdot X_{i+m,j+n} \right) + b$$

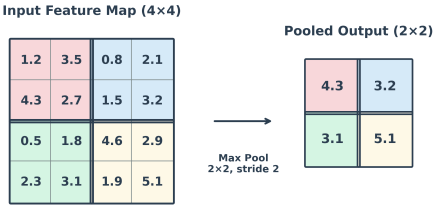
Multiple filters $\Rightarrow$ multiple feature maps, each detecting different patterns.

# Pooling Layers

**Max Pooling** ($2 \times 2$, stride 2):

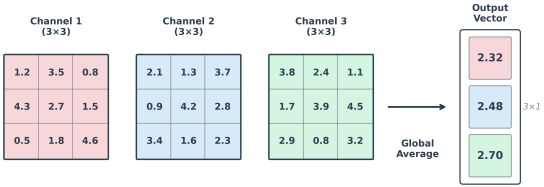
$$Y_{i,j} = \max_{p,q \in \{0,1\}} Z_{2i+p,\, 2j+q}$$

- Reduces spatial dimensions by half.
- Provides local translation invariance.
- Retains strongest activations.



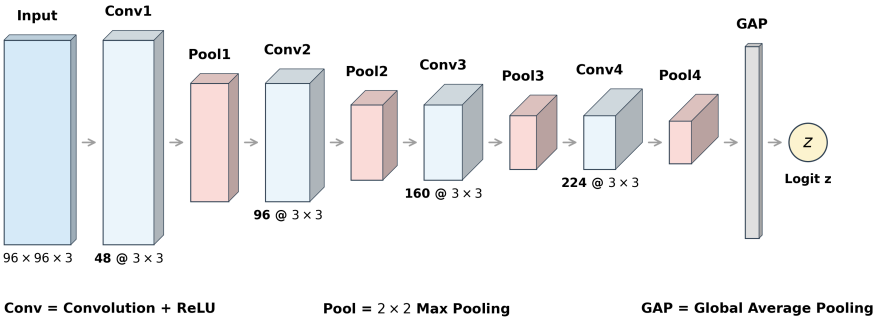

Max pooling on a $4 \times 4$ feature map.

# Global Average Pooling (GAP)




- Computes the **mean** over all spatial positions within each channel.
- Collapses each feature map to a single scalar.
- Drastically reduces parameter count before the classifier.

# CNN Architecture for Crack Detection




- 4 convolutional blocks: 48 $\rightarrow$ 96 $\rightarrow$ 160 $\rightarrow$ 224 filters ($3\times3$, ReLU + $2\times2$ max pool).
- GAP $\rightarrow$ 224-dim vector $\rightarrow$ 1 output logit.
- **Total: 504,321 parameters** (vs. 609,815 for the FNN).

# Accuracy and the Accuracy Paradox

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

**Accuracy paradox:** With class imbalance (e.g. 95% no-crack), predicting "no crack" for every image gives 95% accuracy while detecting zero cracks.

Our dataset is **balanced** (50/50), so accuracy remains a valid metric here.

# Confusion Matrix, Precision, and Recall

| | **Pred. Negative** | **Pred. Positive** |
|---|---|---|
| **Actual Negative** | True Negative (TN) | False Positive (FP) |
| **Actual Positive** | False Negative (FN) | True Positive (TP) |

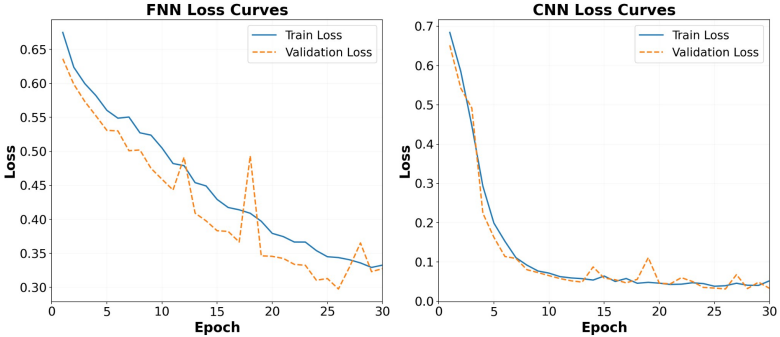
$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Precision "If the AI flags a crack, is it really a crack?"

Recall "Out of all real cracks, how many did the AI find?"

In structural inspection, **recall is typically prioritized** — missing a crack (FN) is more dangerous than a false alarm (FP).

# Loss Curves: FNN vs. CNN




Training and validation loss for FNN (left) and CNN (right).

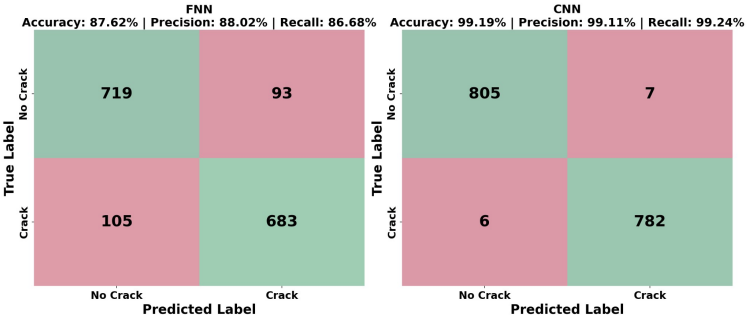
CNN converges faster and reaches lower final loss. FNN shows a larger train–val gap.

# Model Comparison

| Model | Test Accuracy | Test Loss |
|---|---|---|
| FNN (2 hidden layers) | 87.62% | 0.3277 |
| CNN (4 conv + GAP) | **99.19%** | **0.0328** |

- CNN dramatically outperforms FNN on this image task.
- **Why?** CNN exploits spatial structure (edges, textures, crack patterns) through convolutions and weight sharing. FNN treats each pixel independently.

# Confusion Matrices




FNN (left) vs. CNN (right). CNN produces far fewer misclassifications.

# Overfitting in Deep Learning

**Overfitting** = the model fits training noise and fails to generalize.

Common indicators:

- Training loss keeps decreasing while validation loss *increases*.
- Persistent gap between training and validation accuracy.

**Note (double descent):** Increasing model size does not always make generalization worse. In highly over-parameterized regimes, test error can decrease again.

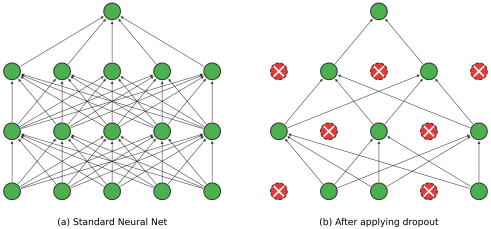
# Regularization Techniques

## Data Augmentation

- Random rotations, flips, brightness changes.
- Exposes the network to more variation.

## Dropout

- Randomly set a fraction of activations to zero during training.
- Prevents co-adaptation of units.




(a) Standard network. (b) Dropout applied.

# More Regularization Techniques

**Early Stopping:**

- Monitor validation loss during training.
- Stop when it has not improved for a specified number of epochs (*patience*).
- Restore the best checkpoint.

**Weight Decay (L2 Regularization):**

$$J_{\text{regularized}} = J + \lambda \sum_k w_k^2$$

- Discourages large weights $\Rightarrow$ simpler, more generalizable models.
- $\lambda$ controls regularization strength.

# Key Takeaways

1. "Training" is **parameter estimation**: define architecture, minimize loss on data.
2. **CNNs** exploit spatial structure and dramatically outperform FNNs on image tasks.
3. The **convolution operation** learns local features; pooling provides downsampling and invariance.
4. Use **cross-entropy** for classification, **MSE** for regression.
5. Choose metrics that match **decision cost**: in safety-critical applications, prioritize recall.
6. Apply **regularization** (augmentation, dropout, early stopping, weight decay) to prevent overfitting.